

流程控制（下）：循环结构

本章学习目标

- 掌握 for 循环的多种形式和使用场景
 - 熟练使用 while 循环处理动态条件
 - 理解 until 循环的适用场景
 - 掌握循环控制语句 break 和 continue
 - 学会使用 select 创建交互式菜单
 - 能够处理嵌套循环和复杂循环逻辑
 - 掌握循环的性能优化技巧
-

6.1 for 循环

for 循环是最常用的循环结构，用于遍历一组值或执行固定次数的操作。

6.1.1 for-in 循环基本语法

```
for variable in list
do
    commands
done
```

或者写成单行形式：

```
for variable in list; do commands; done
```

【实操练习 6-1】for 循环基础示例

```
#!/bin/bash
# 文件名: for_basic.sh
```

```
# 遍历字符串列表
echo "=== 遍历水果列表 ==="
for fruit in apple banana orange grape
do
    echo "水果: $fruit"
done

# 遍历数字序列
echo -e "\n=== 遍历数字 ==="
for num in 1 2 3 4 5
do
    echo "数字: $num"
done

# 遍历带空格的字符串（使用引号）
echo -e "\n=== 遍历文件名 ==="
for file in "my file.txt" "another file.doc" "third file.pdf"
do
    echo "文件: $file"
done
```

6.1.2 遍历各种类型的列表

1. 使用大括号扩展

```
#!/bin/bash

# 数字范围
echo "=== 数字范围 {1..5} ==="
for i in {1..5}
do
    echo "数字: $i"
done

# 带步进的范围（Bash 4.0+）
echo -e "\n=== 带步进 {0..10..2} ==="
for i in {0..10..2}
do
    echo "偶数: $i"
done
```

```
# 字母范围
echo -e "\n=== 字母范围 {a..f} ==="
for letter in {a..f}
do
    echo "字母: $letter"
done

# 组合扩展
echo -e "\n=== 组合扩展 ==="
for item in file{1..3}.txt
do
    echo "文件: $item"
done
```

2. 使用命令输出

```
#!/bin/bash

# 遍历命令输出
echo "=== 遍历当前目录文件 ==="
for file in $(ls *.sh 2>/dev/null)
do
    echo "脚本: $file"
done

# 遍历命令行参数
echo -e "\n=== 遍历 /etc 下的配置文件 ==="
for conf in $(find /etc -maxdepth 1 -name "*.conf" 2>/dev/null | head -5)
do
    echo "配置文件: $conf"
done
```

 **注意：**使用 `$(ls)` 或 `$(find)` 遍历文件名时，如果文件名包含空格会出问题。更安全的方式是使用通配符或 `find ... -exec`。

3. 使用通配符（推荐用于文件遍历）

```
#!/bin/bash

# 遍历当前目录的 .sh 文件
echo "=== 使用通配符遍历 ==="
for script in *.sh
do
    # 检查是否真的有匹配的文件
    if [ -f "$script" ]; then
        echo "脚本: $script ($(wc -l < "$script") 行)"
    fi
done

# 遍历多种类型文件
echo -e "\n=== 遍历多种文件 ==="
for file in /etc/*.conf /etc/*.cfg 2>/dev/null
do
    if [ -f "$file" ]; then
        echo "配置: $(basename "$file")"
    fi
done
```

4. 遍历数组

```
#!/bin/bash

# 定义数组
fruits=("apple" "banana" "cherry" "date")

# 遍历数组元素
echo "=== 遍历数组 ==="
for fruit in "${fruits[@]}"
do
    echo "水果: $fruit"
done

# 遍历数组索引
echo -e "\n=== 遍历数组索引和值 ==="
for i in "${!fruits[@]}"
do
    echo "索引 $i: ${fruits[$i]}"
done
```

6.1.3 C 风格的 for 循环

Bash 还支持类似 C 语言的 for 循环语法：

```
for (( initialization; condition; increment ))
do
    commands
done
```

【实操练习 6-2】C 风格 for 循环

```
#!/bin/bash
# 文件名: for_c_style.sh

# 基本计数
echo "=== 基本计数 ==="
for (( i=1; i<=5; i++ ))
do
    echo "计数: $i"
done

# 倒计数
echo -e "\n=== 倒计数 ==="
for (( i=5; i>=1; i-- ))
do
    echo "倒数: $i"
done

# 自定义步进
echo -e "\n=== 步进为3 ==="
for (( i=0; i<=15; i+=3 ))
do
    echo "数值: $i"
done

# 多个变量
echo -e "\n=== 多变量循环 ==="
for (( i=0, j=10; i<=10; i++, j-- ))
do
```

```
    echo "i=$i, j=$j, sum=$((i+j))"
done

# 无限循环（需要内部 break）
# for (( ; ; ))
# do
#     echo "无限循环..."
#     break
# done
```

6.2 while 循环

while 循环在条件为真时重复执行，适合处理不确定次数的循环。

6.2.1 while 循环基本语法

```
while condition
do
    commands
done
```

【实操练习 6-3】while 循环基础

```
#!/bin/bash
# 文件名: while_basic.sh

# 基本计数循环
echo "=== 基本计数 ==="
count=1
while [ $count -le 5 ]
do
    echo "计数: $count"
    ((count++))
done

# 使用 (( )) 进行条件判断
echo -e "\n=== 使用双括号 ==="
num=10
```

```

while (( num > 0 ))
do
    echo "倒计时: $num"
    ((num--))
done

# 条件为命令的返回值
echo -e "\n=== 等待文件创建 ==="
# 创建测试场景
rm -f /tmp/test_flag
( sleep 3 && touch /tmp/test_flag ) & # 后台3秒后创建文件

timeout=10
elapsed=0
while [ ! -f /tmp/test_flag ] && [ $elapsed -lt $timeout ]
do
    echo "等待文件... ($elapsed 秒)"
    sleep 1
    ((elapsed++))
done

if [ -f /tmp/test_flag ]; then
    echo "文件已创建! "
    rm -f /tmp/test_flag
else
    echo "等待超时! "
fi

```

6.2.2 读取文件内容

while 循环配合 read 命令是读取文件的最佳方式：

```

#!/bin/bash
# 文件名: while_read_file.sh

# 创建测试文件
cat > /tmp/test_data.txt << 'EOF'
apple 10 red
banana 20 yellow
cherry 15 red

```

```
date 8 brown
EOF

# 方法1: 重定向方式
echo "=== 方法1: 重定向 ==="
while read line
do
    echo "行内容: $line"
done < /tmp/test_data.txt

# 方法3: 读取多个字段
echo -e "\n=== 方法3: 读取多个字段 ==="
while read fruit count color
do
    echo "水果: $fruit, 数量: $count, 颜色: $color"
done < /tmp/test_data.txt

# 方法4: 使用自定义分隔符
echo -e "\n=== 方法4: 处理 /etc/passwd ==="
while IFS=':' read user pass uid gid info home shell
do
    if [ "$uid" -lt 1000 ] 2>/dev/null && [ "$uid" -gt 0 ]; then
        echo "系统用户: $user (UID: $uid)"
    fi
done < /etc/passwd | head -5

# 清理
rm -f /tmp/test_data.txt
```

6.2.3 处理命令输出


```
#!/bin/bash
# 文件名: while_command.sh

# 处理 ps 命令输出
echo "=== 当前 bash 进程 ==="
while read user pid cpu mem vsz rss tty stat start time cmd
do
    echo "PID: $pid, CPU: $cpu%, MEM: $mem%"
done < <(ps aux | grep bash)
```

6.3 until 循环

until 循环与 while 相反，在条件为假时执行，条件为真时退出。

6.3.1 until 循环基本语法

```
until condition
do
    commands
done
```

【实操练习 6-4】until 循环

```
#!/bin/bash
# 文件名: until_basic.sh

# 基本示例：等待条件满足
echo "=== 倒计时 ==="
count=5
until [ $count -eq 0 ]
do
    echo "倒计时: $count"
    ((count--))
    sleep 1
done
echo "发射! "
```

```
# 等待服务启动
echo -e "\n=== 等待服务就绪（模拟） ==="
ready=false
attempts=0
max_attempts=5

until $ready || [ $attempts -ge $max_attempts ]
do
    ((attempts++))
    echo "尝试连接... (第 $attempts 次)"

    # 模拟：第3次成功
    if [ $attempts -eq 3 ]; then
        ready=true
    fi

    sleep 1
done

if $ready; then
    echo "服务已就绪! "
else
    echo "连接失败，已达最大尝试次数"
fi

# 等待用户输入正确答案
echo -e "\n=== 猜数字游戏 ==="
secret=7
guess=-1

until [ "$guess" -eq "$secret" ] 2>/dev/null
do
    read -p "猜一个1-10之间的数字: " guess

    if ! [[ "$guess" =~ ^[0-9]+$ ]]; then
        echo "请输入数字! "
        guess=-1
        continue
    fi

    if [ "$guess" -lt "$secret" ]; then
        echo "太小了! "
```

```
    elif [ "$guess" -gt "$secret" ]; then
        echo "太大了! "
    fi
done

echo "恭喜, 猜对了! "
```

6.3.2 while vs until

特性	while	until
循环条件	条件为真时循环	条件为假时循环
退出条件	条件变为假时退出	条件变为真时退出
适用场景	"当...时继续"	"直到...为止"
常见用途	处理数据、读取文件	等待条件满足

```
# 等价的 while 和 until
# while 形式
while [ ! -f /tmp/flag ]; do
    echo "等待..."
    sleep 1
done

# until 形式 (更直观)
until [ -f /tmp/flag ]; do
    echo "等待..."
    sleep 1
done
```

6.4 循环控制语句

6.4.1 break 语句

break 用于立即退出循环

```
#!/bin/bash
# 文件名: break_example.sh

# 基本 break
echo "=== 基本 break ==="
for i in {1..10}
do
    if [ $i -eq 5 ]; then
        echo "在 $i 处中断"
        break
    fi
    echo "处理: $i"
done
echo "循环结束"
```

6.4.2 continue 语句

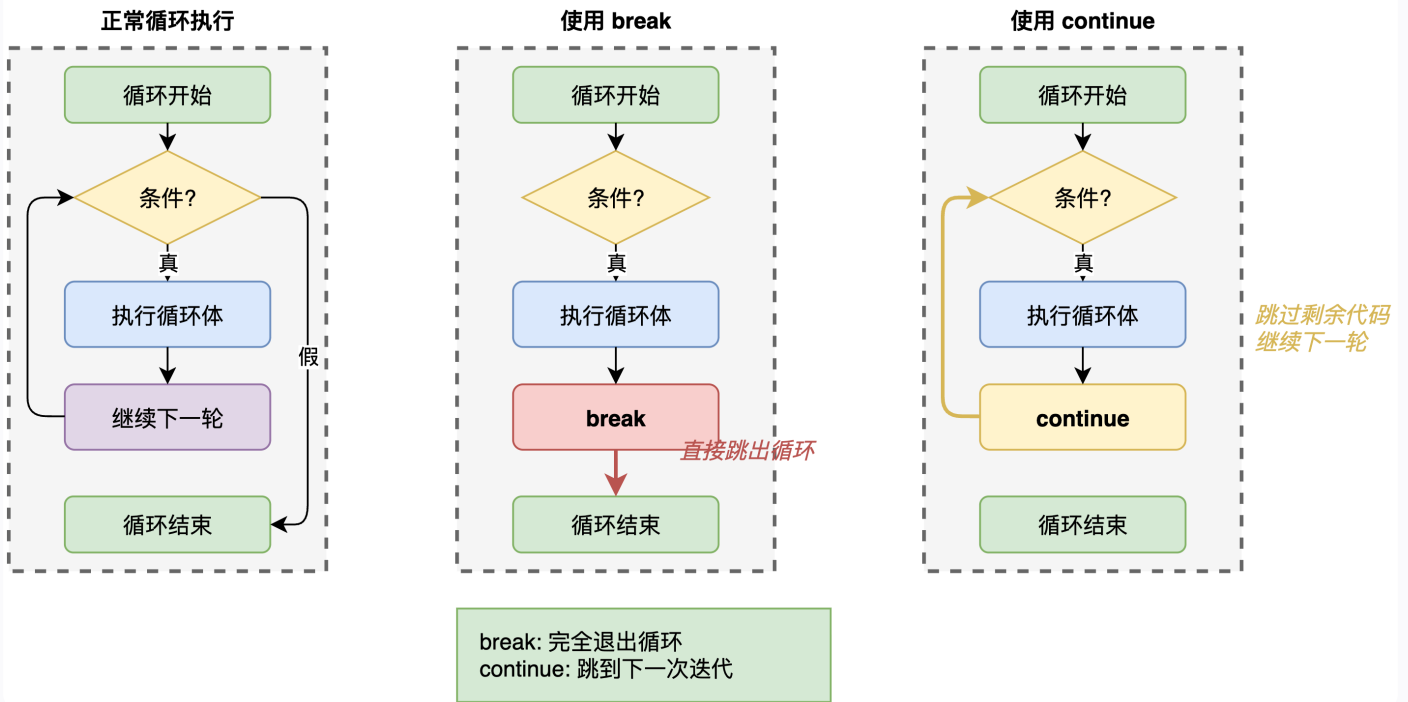
continue 用于跳过当前迭代，继续下一次循环。

```
#!/bin/bash
# 文件名: continue_example.sh

# 基本 continue
echo "=== 跳过偶数 ==="
for i in {1..10}
do
    if (( i % 2 == 0 )); then
        continue # 跳过偶数
    fi
    echo "奇数: $i"
done
```

【图6-2：break 和 continue 执行流程图】

break 和 continue 的区别



6.6 嵌套循环

嵌套循环是指在一个循环内部包含另一个循环。

6.6.1 嵌套循环基础

```
#!/bin/bash
# 文件名: nested_loops.sh

# 打印乘法表
echo "=== 九九乘法表 ==="
for i in {1..9}
do
    for j in $(seq 1 $i)
    do
        printf "%d×%d=%-2d  " $j $i $((i*j))
    done
    echo ""
done

# 遍历二维数组概念
echo -e "\n=== 遍历多个目录 ==="
```

```

dirs=("/etc" "/usr" "/var")
extensions=("conf" "log")

for dir in "${dirs[@]}"
do
    echo "检查目录: $dir"
    for ext in "${extensions[@]}"
    do
        count=$(find "$dir" -maxdepth 2 -name ".*$ext" 2>/dev/null |
wc -l)
        echo "    .$ext 文件数量: $count"
    done
done

```

6.9 循环最佳实践

6.9.1 选择合适的循环类型

场景	推荐循环	原因
遍历已知列表	for-in	语法简洁
固定次数循环	for C风格 或 for {1..n}	明确迭代次数
读取文件内容	while read	逐行处理
条件不确定的循环	while	动态判断
等待条件满足	until	语义更清晰

6.9.2 性能优化技巧

```

#!/bin/bash

# 1. 避免在循环中重复计算
# 不好

```

```
for file in *.txt; do
    echo "$(date): 处理 $file" # 每次都调用 date
done
```

```
# 好
current_time=$(date) # 只调用一次
for file in *.txt; do
    echo "$current_time: 处理 $file"
done
```

2. 减少子进程创建

```
# 不好
for i in $(seq 1 1000); do
    echo "$i"
done
```

```
# 好
for i in {1..1000}; do
    echo "$i"
done
```

3. 使用内置命令替代外部命令

```
# 不好
length=$(echo "$string" | wc -c)
```

```
# 好
length=${#string}
```

4. 批量操作替代循环

不好（对每个文件单独调用 chmod）

```
for file in *.sh; do
    chmod +x "$file"
done
```

好（一次调用处理所有文件）

```
chmod +x *.sh 2>/dev/null
```

5. 使用数组存储中间结果

```
declare -a results
for i in {1..10}; do
    results+=("结果$i")
done
```

```
printf '%s\n' "${results[@]}"
```

6.10 本章小结

本章我们学习了 Shell 脚本中的循环结构：

for 循环：

1. for-in 形式遍历列表、数组、文件
2. C 风格形式用于计数循环
3. 支持大括号扩展 `{1..10}` 和 `{a..z}`

while 循环：

1. 条件为真时持续执行
2. 配合 read 读取文件是最佳实践
3. 注意管道循环的变量作用域问题

until 循环：

1. 条件为假时持续执行，条件为真时退出
2. 适合"等待直到..."的场景

循环控制：

1. break 退出循环，可指定退出层数
2. continue 跳过当前迭代

最佳实践：

1. 选择合适的循环类型
2. 避免在循环内重复执行耗时操作
3. 正确处理包含空格的文件名
4. 注意管道循环的变量作用域

